

Research Note — LLD Practice Platform

Problem understanding, feedback strategy, evaluation design, and MVP decisions

1. Problem I Was Trying to Solve

Low-Level Design is different from a problem where there is one expected output. For the same requirement, two developers can produce different class structures and both can be reasonable if their responsibilities, relationships, and trade-offs make sense.

That makes practice difficult. A learner can finish a Parking Lot or Elevator design, but simply knowing that a submission was accepted does not explain whether the design is cohesive, whether responsibilities are placed well, or whether important requirements were represented.

My focus became: How can a practice platform help a learner understand why their LLD approach may be weak, instead of only telling them whether it is right or wrong?

Practice loop: Choose a problem → Design → Submit → Get feedback → Review → Try again.

2. What I Found Important for LLD Practice

There is rarely one “correct” design. LLD is largely about responsibilities, relationships, abstraction, coupling, cohesion, extensibility, and trade-offs. A platform should therefore not compare every submission against one fixed class diagram.

I treat each problem's requirements, constraints, and expected concepts as guidance. Expected concepts are signals to consider, not mandatory answers.

Feedback needs to be specific and actionable. “Your design can be improved” is weak feedback. The learner needs to know what was observed and what they can reconsider.

3. Why I Did Not Make the LLM the Only Evaluator

Structural — High confidence: deterministic checks for missing class names, missing responsibilities, invalid relationships, and basic submission structure.

Heuristic — Medium confidence: signals such as whether important requirement concepts appear in the submission. These are presented as “Worth Considering,” not as pass/fail results.

AI — Contextual: Gemini receives the problem, submission, and deterministic findings. It can reason about abstraction, coupling, cohesion, requirement interpretation, and trade-offs.

This hybrid approach was intentional. Research on LLMs for software engineering highlights their usefulness as well as reliability and hallucination concerns.

4. What the Learner Should Provide

I considered a completely free-form submission, but consistent evaluation would be difficult within the MVP scope.

The MVP uses a structured submission: **Classes, Responsibilities, Relationships, Patterns Used, and an Optional Code Stub**. This gives the evaluator machine-readable information while still letting the learner explain the design.

The optional code stub adds context for AI feedback without requiring code execution or a sandbox. The submission model can also support additional formats later.

5. Product and Engineering Decisions

I deliberately kept the MVP as a **monolith**. The main problem is the learner experience and evaluation model, so microservices, queues, Kubernetes, and other distributed infrastructure would add complexity without improving the core learning loop.

I made feedback confidence visible: **High confidence** for deterministic checks, **Medium confidence** for heuristic observations, and **Contextual** for AI insights.

If the AI provider fails, deterministic and heuristic feedback can still be returned. The UI explicitly communicates that AI feedback is unavailable rather than silently hiding the limitation.

6. What I Would Explore Next

Future work could include richer diagrams, stronger semantic requirement coverage, more problem types, calibrated evaluation using manually reviewed examples, comparison of multiple valid designs, deeper trade-off feedback, and additional submission formats.

The biggest future challenge is evaluation quality. LLD is subjective enough that a good system should improve through examples and calibration rather than simply adding more rules or making the LLM prompt longer.

References

1. Martin Fowler — *Layering Principles*. <https://martinfowler.com/bliki/LayeringPrinciples.html>
2. Martin Fowler — *Reducing Coupling*. <https://martinfowler.com/ieeeSoftware/coupling.pdf>
3. Fan et al. — *Large Language Models for Software Engineering: Survey and Open Problems*. <https://arxiv.org/abs/2310.03533>
4. Frontiers in Education (2025) — *Student support during project-based learning using simulated automated formative feedback*.